Structured Analysis of Long Technical Documents

Technical Report TR-2026-114

Document Intelligence Group, Northgate University

Abstract

This report documents the design, implementation and evaluation of a pipeline that recovers the structured outline of long technical documents. The report is organized as a deeply nested document: chapters are numbered in the 1 style, sections in the 1.1 style, and subsections in the 1.1.1 style, so that a parser must reconstruct three levels of hierarchy from typography alone. Each subsection contains genuinely distinct prose drawn from the vocabulary of its own subproblem, which makes the fixture useful for testing heading detection, hierarchy induction and long-document chunking under realistic conditions. The evaluation in Chapter 4 reports section boundary F1, heading level accuracy and outline depth exact match for four reference systems, and the final two pages contain the bibliography that a reference parser is expected to segment.

Keywords: long document parsing; outline recovery; heading detection; hierarchy induction; technical reports

1 Introduction

In this section we describe layout segmentation. The design goal is to make layout segmentation reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, the tuned configuration uses a window of 14 units, a stride of 9 units and a confidence floor of 0.8. The implementation of layout segmentation follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, the fallback path is triggered for roughly 32 percent of pages, mostly scanned documents with a damaged text layer. An important design decision concerns the granularity at which layout segmentation operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, the tuned configuration uses a window of 10 units, a stride of 24 units and a confidence floor of 0.2. We report the behaviour of layout segmentation on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, the fallback path is triggered for roughly 36 percent of pages, mostly scanned documents with a damaged text layer.

1.1 Motivation and Scope

The implementation of table extraction follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 18 steps and early stopping patience of 5 epochs. In this section we describe table extraction. The design goal is to make table extraction reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, error bars are obtained by bootstrapping 12 resamples of the evaluation set, reported as a 13 percent confidence interval. We report the behaviour of table extraction on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 38 steps and early stopping patience of 25 epochs. In this section we describe table extraction. The design goal is to make table extraction reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, the tuned configuration uses a window of 32 units, a stride of 19 units and a confidence floor of 0.6.

1.1.1 Cell Grid Reconstruction

The cost of numeric column typing is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, we cap the candidate list at 39 entries and truncate each entry to 22 characters before scoring. In this section we describe numeric column typing. The design goal is to make numeric column typing reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, memory usage stays below 13 megabytes per worker because intermediate representations are streamed rather than materialized.

We report the behaviour of numeric column typing on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, the tuned configuration uses a window of 11 units, a stride of 21 units and a confidence floor of 0.8. Our first prototype of numeric column typing relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, annotation guidelines were revised 25 times, and inter annotator agreement settled at kappa equal to 0.1. Empirically, numeric column typing accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, error bars are obtained by bootstrapping 32 resamples of the evaluation set, reported as a 10 percent confidence interval.

1.1.2 Spanning Cell Handling

Our first prototype of spanning cell handling relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, the tuned configuration uses a window of 8 units, a stride of 5 units and a confidence floor of 0.2. We report the behaviour of spanning cell handling on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 14 steps and early stopping patience of 14 epochs. Our first prototype of spanning cell handling relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, the default threshold is set to 0.8, chosen on the development split to maximize the harmonic mean of precision and recall. We report the behaviour of spanning cell handling on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, a single pass over the corpus of 42 documents completes in 4 minutes on eight worker processes without GPU acceleration. We evaluated spanning cell handling against two reference implementations. The comparison is reported in the accompanying result tables and discussed in the analysis subsection below. Specifically, the tuned configuration uses a window of 7 units, a stride of 15 units and a confidence floor of 0.6.

1.2 Problem Statement

Because formula recognition interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, a single pass over the corpus of 9 documents completes in 24 minutes on eight worker processes without GPU acceleration. We treat formula recognition as a bounded subproblem of the overall parsing pipeline. formula recognition receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, we cap the candidate list at 19 entries and truncate each entry to 9 characters before scoring. The current version of formula recognition is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 5 steps and early stopping patience of 12 epochs. We treat formula recognition as a bounded subproblem of the overall parsing pipeline. formula recognition receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, we cap the candidate list at 31 entries and truncate each entry to 10 characters before scoring.

1.2.1 Inline Formula Isolation

The cost of display formula grouping is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, a single pass over the corpus of 15 documents completes in 24 minutes on eight worker processes without GPU acceleration. Our first prototype of display formula grouping relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, error bars are obtained by bootstrapping 5 resamples of the evaluation set, reported as a 16 percent confidence interval. The implementation of display formula grouping follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, the fallback path is triggered for roughly 35 percent of pages, mostly scanned documents with a damaged text layer. Our first prototype of display formula grouping relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, the default threshold is set to 0.6, chosen on the development split to maximize the harmonic mean of precision and recall. Empirically, display formula grouping accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, we fixed the learning rate schedule

to cosine decay with a warmup of 8 steps and early stopping patience of 27 epochs.

1.2.2 Display Formula Grouping

In this section we describe inline formula isolation. The design goal is to make inline formula isolation reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 40 steps and early stopping patience of 12 epochs. We report the behaviour of inline formula isolation on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, we cap the candidate list at 14 entries and truncate each entry to 22 characters before scoring. An important design decision concerns the granularity at which inline formula isolation operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 12 steps and early stopping patience of 20 epochs. The cost of inline formula isolation is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.6 are treated as meaningful. We treat inline formula isolation as a bounded subproblem of the overall parsing pipeline. inline formula isolation receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, memory usage stays below 7 megabytes per worker because intermediate representations are streamed rather than materialized.

The current version of inline formula isolation is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, a single pass over the corpus of 40 documents completes in 12 minutes on eight worker processes without GPU acceleration. We treat inline formula isolation as a bounded subproblem of the overall parsing pipeline. inline formula isolation receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, error bars are obtained by bootstrapping 14 resamples of the evaluation set, reported as a 22 percent confidence interval. Empirically, inline formula isolation accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 12 steps and early stopping patience of 20 epochs.

1.3 Contributions

The current version of reference parsing is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, annotation guidelines were revised 28 times, and inter annotator agreement settled at kappa equal to 0.5. We treat reference parsing as a bounded subproblem of the overall parsing pipeline. reference parsing receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, we cap the candidate list at 26 entries and truncate each entry to 28 characters before scoring. Because reference parsing interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 40 steps and early stopping patience of 11 epochs. A recurring difficulty for reference parsing is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.6 are treated as meaningful.

1.3.1 Reference Block Segmentation

We treat reference block segmentation as a bounded subproblem of the overall parsing pipeline. reference block segmentation receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, memory usage stays below 31 megabytes per worker because intermediate representations are streamed rather than materialized. Because reference block segmentation interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.8 are treated as meaningful. We treat reference block segmentation as a bounded subproblem of the overall parsing pipeline. reference block segmentation receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, a single pass over the corpus of 19 documents completes in 25 minutes on eight worker

processes without GPU acceleration. The current version of reference block segmentation is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, error bars are obtained by bootstrapping 41 resamples of the evaluation set, reported as a 11 percent confidence interval. In this section we describe reference block segmentation. The design goal is to make reference block segmentation reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, memory usage stays below 8 megabytes per worker because intermediate representations are streamed rather than materialized.

1.3.2 Field Level Tagging

We report the behaviour of venue normalization on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, a single pass over the corpus of 16 documents completes in 18 minutes on eight worker processes without GPU acceleration. Our first prototype of venue normalization relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, the default threshold is set to 0.8, chosen on the development split to maximize the harmonic mean of precision and recall. The cost of venue normalization is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 28 steps and early stopping patience of 12 epochs. Our first prototype of venue normalization relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, the tuned configuration uses a window of 10 units, a stride of 24 units and a confidence floor of 0.2. Empirically, venue normalization accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, annotation guidelines were revised 7 times, and inter annotator agreement settled at kappa equal to 0.5.

1.4 Report Organization

Because metadata reconciliation interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, annotation guidelines were revised 15 times, and inter annotator agreement settled at kappa equal to 0.5. A recurring difficulty for metadata reconciliation is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, the tuned configuration uses a window of 29 units, a stride of 12 units and a confidence floor of 0.4. Because metadata reconciliation interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, a single pass over the corpus of 3 documents completes in 20 minutes on eight worker processes without GPU acceleration. We treat metadata reconciliation as a bounded subproblem of the overall parsing pipeline. metadata reconciliation receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, the tuned configuration uses a window of 33 units, a stride of 28 units and a confidence floor of 0.6.

1.4.1 Title Matching

The current version of duplicate merging is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, the fallback path is triggered for roughly 15 percent of pages, mostly scanned documents with a damaged text layer. We treat duplicate merging as a bounded subproblem of the overall parsing pipeline. duplicate merging receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, we cap the candidate list at 29 entries and truncate each entry to 7 characters before scoring. The current version of duplicate merging is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, annotation guidelines were revised 3 times, and inter annotator agreement settled at kappa equal to 0.1. We treat duplicate merging as a bounded subproblem of the overall parsing pipeline. duplicate merging receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, the tuned configuration uses a window of 41 units, a stride of 2 units and a confidence floor of 0.8. The cost of duplicate merging is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, memory usage stays below 32 megabytes per worker because intermediate representations are streamed rather than materialized.

Our first prototype of duplicate merging relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, memory usage stays below 15 megabytes per worker because intermediate representations are streamed rather than materialized. We report the behaviour of duplicate merging on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.2 are treated as meaningful. An important design decision concerns the granularity at which duplicate merging operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, a single pass over the corpus of 3 documents completes in 23 minutes on eight worker processes without GPU acceleration.

1.4.2 Author Affiliation Binding

Because venue string canonicalization interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, memory usage stays below 24 megabytes per worker because intermediate representations are streamed rather than materialized. A recurring difficulty for venue string canonicalization is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, the default threshold is set to 0.2, chosen on the development split to maximize the harmonic mean of precision and recall. The current version of venue string canonicalization is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, memory usage stays below 4 megabytes per worker because intermediate representations are streamed rather than materialized. We evaluated venue string canonicalization against two reference implementations. The comparison is reported in the accompanying result tables and discussed in the analysis subsection below. Specifically, the tuned configuration uses a window of 34 units, a stride of 17 units and a confidence floor of 0.8. The implementation of venue string canonicalization follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 31 steps and early stopping patience of 27 epochs.

2 System Architecture

We report the behaviour of reading order recovery on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, error bars are obtained by bootstrapping 5 resamples of the evaluation set, reported as a 26 percent confidence interval. An important design decision concerns the granularity at which reading order recovery operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, memory usage stays below 39 megabytes per worker because intermediate representations are streamed rather than materialized. The cost of reading order recovery is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, error bars are obtained by bootstrapping 41 resamples of the evaluation set, reported as a 9 percent confidence interval. In this section we describe reading order recovery. The design goal is to make reading order recovery reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, annotation guidelines were revised 11 times, and inter annotator agreement settled at kappa equal to 0.7.

2.1 Pipeline Overview

An important design decision concerns the granularity at which cross-page boilerplate filtering operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, the tuned configuration uses a window of 10 units, a stride of 3 units and a confidence floor of 0.2. The cost of cross-page boilerplate filtering is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, memory usage stays below 4 megabytes per worker because intermediate representations are streamed rather than materialized. An important design decision concerns the granularity at which cross-page boilerplate filtering operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, the default threshold is set to 0.8, chosen on the development split to maximize the harmonic mean of precision and recall. The cost of cross-page boilerplate filtering is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically,

a single pass over the corpus of 24 documents completes in 9 minutes on eight worker processes without GPU acceleration.

2.1.1 Repetition Statistics

A recurring difficulty for page number templating is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, annotation guidelines were revised 28 times, and inter annotator agreement settled at kappa equal to 0.5. The current version of page number templating is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, the default threshold is set to 0.4, chosen on the development split to maximize the harmonic mean of precision and recall. A recurring difficulty for page number templating is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, a single pass over the corpus of 32 documents completes in 4 minutes on eight worker processes without GPU acceleration. Empirically, page number templating accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, error bars are obtained by bootstrapping 22 resamples of the evaluation set, reported as a 23 percent confidence interval. In this section we describe page number templating. The design goal is to make page number templating reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, the fallback path is triggered for roughly 27 percent of pages, mostly scanned documents with a damaged text layer.

2.1.2 Margin Band Detection

A recurring difficulty for margin band detection is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, a single pass over the corpus of 35 documents completes in 2 minutes on eight worker processes without GPU acceleration. Because margin band detection interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, the tuned configuration uses a window of 33 units, a stride of 15 units and a confidence floor of 0.4. We treat margin band detection as a bounded subproblem of the overall parsing pipeline. margin band detection receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 23 steps and early stopping patience of 27 epochs. Because margin band detection interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, the tuned configuration uses a window of 21 units, a stride of 12 units and a confidence floor of 0.6. Our first prototype of margin band detection relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, annotation guidelines were revised 28 times, and inter annotator agreement settled at kappa equal to 0.1.

2.2 Interface Contracts

Because multi-column flow analysis interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.2 are treated as meaningful. We treat multi-column flow analysis as a bounded subproblem of the overall parsing pipeline. multi-column flow analysis receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, a single pass over the corpus of 35 documents completes in 10 minutes on eight worker processes without GPU acceleration. Because multi-column flow analysis interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, error bars are obtained by bootstrapping 13 resamples of the evaluation set, reported as a 14 percent confidence interval. We treat multi-column flow analysis as a bounded subproblem of the overall parsing pipeline. multi-column flow analysis receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, memory usage stays below 23 megabytes per worker because intermediate representations are streamed rather than materialized.

2.2.1 Gutter Detection

The implementation of column continuation tracking follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora.

Specifically, the tuned configuration uses a window of 12 units, a stride of 8 units and a confidence floor of 0.2. An important design decision concerns the granularity at which column continuation tracking operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, a single pass over the corpus of 10 documents completes in 28 minutes on eight worker processes without GPU acceleration. The cost of column continuation tracking is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, the tuned configuration uses a window of 24 units, a stride of 5 units and a confidence floor of 0.8. In this section we describe column continuation tracking. The design goal is to make column continuation tracking reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, annotation guidelines were revised 38 times, and inter annotator agreement settled at kappa equal to 0.1. Empirically, column continuation tracking accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, error bars are obtained by bootstrapping 19 resamples of the evaluation set, reported as a 13 percent confidence interval.

2.2.2 Column Continuation Tracking

The implementation of gutter detection follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, the default threshold is set to 0.2, chosen on the development split to maximize the harmonic mean of precision and recall. Our first prototype of gutter detection relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, memory usage stays below 17 megabytes per worker because intermediate representations are streamed rather than materialized. We report the behaviour of gutter detection on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, we cap the candidate list at 23 entries and truncate each entry to 18 characters before scoring. In this section we describe gutter detection. The design goal is to make gutter detection reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, the fallback path is triggered for roughly 13 percent of pages, mostly scanned documents with a damaged text layer. Empirically, gutter detection accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, we cap the candidate list at 4 entries and truncate each entry to 12 characters before scoring.

A recurring difficulty for gutter detection is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, error bars are obtained by bootstrapping 27 resamples of the evaluation set, reported as a 25 percent confidence interval. Empirically, gutter detection accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, memory usage stays below 17 megabytes per worker because intermediate representations are streamed rather than materialized. A recurring difficulty for gutter detection is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, error bars are obtained by bootstrapping 23 resamples of the evaluation set, reported as a 18 percent confidence interval.

2.3 Data Representations

We evaluated section boundary scoring against two reference implementations. The comparison is reported in the accompanying result tables and discussed in the analysis subsection below. Specifically, the default threshold is set to 0.2, chosen on the development split to maximize the harmonic mean of precision and recall. Because section boundary scoring interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, annotation guidelines were revised 18 times, and inter annotator agreement settled at kappa equal to 0.7. We evaluated section boundary scoring against two reference implementations. The comparison is reported in the accompanying result tables and discussed in the analysis subsection below. Specifically, error bars are obtained by bootstrapping 32 resamples of the evaluation set, reported as a 6 percent confidence interval. Because section boundary scoring interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, annotation guidelines were revised 14 times, and inter annotator agreement settled at kappa equal to 0.1.

2.3.1 Boundary Classifier Training

We report the behaviour of boundary classifier training on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, we cap the candidate list at 12 entries and truncate each entry to 2 characters before scoring. In this section we describe boundary classifier training. The design goal is to make boundary classifier training reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, annotation guidelines were revised 10 times, and inter annotator agreement settled at kappa equal to 0.3. The implementation of boundary classifier training follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, we cap the candidate list at 24 entries and truncate each entry to 6 characters before scoring. Our first prototype of boundary classifier training relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, a single pass over the corpus of 22 documents completes in 15 minutes on eight worker processes without GPU acceleration. Empirically, boundary classifier training accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, the default threshold is set to 0.2, chosen on the development split to maximize the harmonic mean of precision and recall.

Table 2.3.1: boundary classifier training.

System	heading level accuracy (%)	delta
Rule based pipeline	86.5	-
Layout model + heuristics	88.3	+1.8
Proposed pipeline	91.2	+4.7

2.3.2 Confidence Calibration

An important design decision concerns the granularity at which boundary refinement operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, the tuned configuration uses a window of 35 units, a stride of 9 units and a confidence floor of 0.6. We report the behaviour of boundary refinement on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, the fallback path is triggered for roughly 33 percent of pages, mostly scanned documents with a damaged text layer. An important design decision concerns the granularity at which boundary refinement operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, the tuned configuration uses a window of 23 units, a stride of 8 units and a confidence floor of 0.4. We report the behaviour of boundary refinement on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, annotation guidelines were revised 21 times, and inter annotator agreement settled at kappa equal to 0.5. A recurring difficulty for boundary refinement is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, we cap the candidate list at 4 entries and truncate each entry to 14 characters before scoring.

2.4 Failure Handling

In this section we describe outline normalization. The design goal is to make outline normalization reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, the default threshold is set to 0.2, chosen on the development split to maximize the harmonic mean of precision and recall. The implementation of outline normalization follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, a single pass over the corpus of 37 documents completes in 9 minutes on eight worker processes without GPU acceleration. An important design decision concerns the granularity at which outline normalization operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, we cap the candidate list at 3 entries and truncate each entry to 13 characters before scoring. The implementation of outline normalization follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, annotation guidelines were revised 9 times, and inter annotator agreement settled at kappa equal to 0.1.

2.4.1 Numbering Scheme Detection

An important design decision concerns the granularity at which appendix handling operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, the

tuned configuration uses a window of 20 units, a stride of 10 units and a confidence floor of 0.4. The implementation of appendix handling follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, the fallback path is triggered for roughly 10 percent of pages, mostly scanned documents with a damaged text layer. In this section we describe appendix handling. The design goal is to make appendix handling reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.6 are treated as meaningful. The cost of appendix handling is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, a single pass over the corpus of 30 documents completes in 26 minutes on eight worker processes without GPU acceleration. We evaluated appendix handling against two reference implementations. The comparison is reported in the accompanying result tables and discussed in the analysis subsection below. Specifically, the default threshold is set to 0.8, chosen on the development split to maximize the harmonic mean of precision and recall.

Because appendix handling interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, the default threshold is set to 0.4, chosen on the development split to maximize the harmonic mean of precision and recall. A recurring difficulty for appendix handling is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, memory usage stays below 10 megabytes per worker because intermediate representations are streamed rather than materialized. The current version of appendix handling is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, error bars are obtained by bootstrapping 40 resamples of the evaluation set, reported as a 20 percent confidence interval.

2.4.2 Skipped Level Repair

A recurring difficulty for duplicate heading merging is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, the default threshold is set to 0.4, chosen on the development split to maximize the harmonic mean of precision and recall. The current version of duplicate heading merging is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, a single pass over the corpus of 17 documents completes in 17 minutes on eight worker processes without GPU acceleration. We evaluated duplicate heading merging against two reference implementations. The comparison is reported in the accompanying result tables and discussed in the analysis subsection below. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.6 are treated as meaningful. The current version of duplicate heading merging is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, the fallback path is triggered for roughly 5 percent of pages, mostly scanned documents with a damaged text layer. In this section we describe duplicate heading merging. The design goal is to make duplicate heading merging reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, error bars are obtained by bootstrapping 36 resamples of the evaluation set, reported as a 18 percent confidence interval.

3 Core Components

Our first prototype of heading detection relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, we cap the candidate list at 8 entries and truncate each entry to 26 characters before scoring. We report the behaviour of heading detection on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, a single pass over the corpus of 22 documents completes in 4 minutes on eight worker processes without GPU acceleration. In this section we describe heading detection. The design goal is to make heading detection reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, the default threshold is set to 0.2, chosen on the development split to maximize the harmonic mean of precision and recall. The cost of heading detection is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, a single pass over the corpus of 34 documents completes in 9 minutes on eight worker processes without GPU acceleration.

3.1 Layout Processing

In this section we describe long-context chunking. The design goal is to make long-context chunking reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.6 are treated as meaningful. The cost of long-context chunking is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, a single pass over the corpus of 20 documents completes in 15 minutes on eight worker processes without GPU acceleration. An important design decision concerns the granularity at which long-context chunking operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, we cap the candidate list at 38 entries and truncate each entry to 12 characters before scoring. We report the behaviour of long-context chunking on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, memory usage stays below 24 megabytes per worker because intermediate representations are streamed rather than materialized.

3.1.1 Chunk Size Selection

Because state carry over interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, we cap the candidate list at 24 entries and truncate each entry to 28 characters before scoring. We evaluated state carry over against two reference implementations. The comparison is reported in the accompanying result tables and discussed in the analysis subsection below. Specifically, a single pass over the corpus of 22 documents completes in 20 minutes on eight worker processes without GPU acceleration. Empirically, state carry over accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, the default threshold is set to 0.2, chosen on the development split to maximize the harmonic mean of precision and recall. We treat state carry over as a bounded subproblem of the overall parsing pipeline. state carry over receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, memory usage stays below 34 megabytes per worker because intermediate representations are streamed rather than materialized. We report the behaviour of state carry over on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, we cap the candidate list at 39 entries and truncate each entry to 28 characters before scoring.

3.1.2 Overlap Strategy

Because overlap strategy interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, the tuned configuration uses a window of 39 units, a stride of 14 units and a confidence floor of 0.8. A recurring difficulty for overlap strategy is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, annotation guidelines were revised 21 times, and inter annotator agreement settled at kappa equal to 0.1. Because overlap strategy interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, the tuned configuration uses a window of 19 units, a stride of 8 units and a confidence floor of 0.2. A recurring difficulty for overlap strategy is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, a single pass over the corpus of 25 documents completes in 25 minutes on eight worker processes without GPU acceleration. The cost of overlap strategy is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, the tuned configuration uses a window of 24 units, a stride of 28 units and a confidence floor of 0.4.

3.2 Structure Recovery

We evaluated semantic section typing against two reference implementations. The comparison is reported in the accompanying result tables and discussed in the analysis subsection below. Specifically, error bars are obtained by bootstrapping 17 resamples of the evaluation set, reported as a 11 percent confidence interval. Because semantic section typing interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, annotation guidelines were revised 11 times, and inter annotator agreement settled at kappa equal to 0.3. We treat semantic section typing as a bounded subproblem of the overall parsing pipeline. semantic section typing receives a normalized page representation and emits a typed structure together with a confidence value that later stages may

abstain on. Specifically, error bars are obtained by bootstrapping 13 resamples of the evaluation set, reported as a 13 percent confidence interval. The current version of semantic section typing is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, the fallback path is triggered for roughly 15 percent of pages, mostly scanned documents with a damaged text layer.

3.2.1 Section Label Taxonomy

Because weak supervision design interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, memory usage stays below 32 megabytes per worker because intermediate representations are streamed rather than materialized. A recurring difficulty for weak supervision design is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, error bars are obtained by bootstrapping 14 resamples of the evaluation set, reported as a 14 percent confidence interval. The current version of weak supervision design is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, memory usage stays below 28 megabytes per worker because intermediate representations are streamed rather than materialized. A recurring difficulty for weak supervision design is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.4 are treated as meaningful. The implementation of weak supervision design follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, the fallback path is triggered for roughly 39 percent of pages, mostly scanned documents with a damaged text layer.

3.2.2 Weak Supervision Design

We treat section label taxonomy as a bounded subproblem of the overall parsing pipeline. section label taxonomy receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, annotation guidelines were revised 31 times, and inter annotator agreement settled at kappa equal to 0.3. Empirically, section label taxonomy accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, the default threshold is set to 0.6, chosen on the development split to maximize the harmonic mean of precision and recall. We evaluated section label taxonomy against two reference implementations. The comparison is reported in the accompanying result tables and discussed in the analysis subsection below. Specifically, the fallback path is triggered for roughly 3 percent of pages, mostly scanned documents with a damaged text layer. Empirically, section label taxonomy accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, we cap the candidate list at 41 entries and truncate each entry to 14 characters before scoring. An important design decision concerns the granularity at which section label taxonomy operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 32 steps and early stopping patience of 23 epochs.

The cost of section label taxonomy is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, memory usage stays below 31 megabytes per worker because intermediate representations are streamed rather than materialized. Our first prototype of section label taxonomy relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, we cap the candidate list at 13 entries and truncate each entry to 2 characters before scoring. The cost of section label taxonomy is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, a single pass over the corpus of 3 documents completes in 4 minutes on eight worker processes without GPU acceleration.

3.3 Semantic Annotation

The current version of figure caption association is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.6 are treated as meaningful. We evaluated figure caption association against two reference implementations. The comparison is reported in the accompanying result tables and discussed in the analysis subsection below.

Specifically, annotation guidelines were revised 18 times, and inter annotator agreement settled at kappa equal to 0.3. Because figure caption association interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, error bars are obtained by bootstrapping 32 resamples of the evaluation set, reported as a 13 percent confidence interval. A recurring difficulty for figure caption association is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, memory usage stays below 6 megabytes per worker because intermediate representations are streamed rather than materialized.

3.3.1 Caption Pattern Matching

Because caption pattern matching interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, a single pass over the corpus of 8 documents completes in 10 minutes on eight worker processes without GPU acceleration. We evaluated caption pattern matching against two reference implementations. The comparison is reported in the accompanying result tables and discussed in the analysis subsection below. Specifically, we cap the candidate list at 30 entries and truncate each entry to 25 characters before scoring. Empirically, caption pattern matching accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 20 steps and early stopping patience of 24 epochs. We treat caption pattern matching as a bounded subproblem of the overall parsing pipeline. caption pattern matching receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.8 are treated as meaningful. The cost of caption pattern matching is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 39 steps and early stopping patience of 16 epochs.

3.3.2 Spatial Proximity Scoring

In this section we describe credit line stripping. The design goal is to make credit line stripping reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, the fallback path is triggered for roughly 7 percent of pages, mostly scanned documents with a damaged text layer. The cost of credit line stripping is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, the default threshold is set to 0.2, chosen on the development split to maximize the harmonic mean of precision and recall. In this section we describe credit line stripping. The design goal is to make credit line stripping reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, a single pass over the corpus of 27 documents completes in 12 minutes on eight worker processes without GPU acceleration. The implementation of credit line stripping follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, the tuned configuration uses a window of 17 units, a stride of 5 units and a confidence floor of 0.8. A recurring difficulty for credit line stripping is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, the fallback path is triggered for roughly 32 percent of pages, mostly scanned documents with a damaged text layer.

Table 3.3.2: credit line stripping.

System	table cell F1 (%)	delta
Rule based pipeline	84.0	-
Layout model + heuristics	85.8	+1.8
Proposed pipeline	88.4	+4.4

3.4 Quality Control

Our first prototype of footnote separation relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, we cap the candidate list at 23 entries and truncate each entry to 19 characters before scoring. The implementation of footnote separation follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, the fallback path is triggered for

roughly 5 percent of pages, mostly scanned documents with a damaged text layer. Our first prototype of footnote separation relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, the tuned configuration uses a window of 35 units, a stride of 6 units and a confidence floor of 0.4. The implementation of footnote separation follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 25 steps and early stopping patience of 27 epochs.

3.4.1 Marker Detection

Empirically, note reattachment accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, the fallback path is triggered for roughly 40 percent of pages, mostly scanned documents with a damaged text layer. We treat note reattachment as a bounded subproblem of the overall parsing pipeline. note reattachment receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.8 are treated as meaningful. Empirically, note reattachment accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 28 steps and early stopping patience of 23 epochs. We treat note reattachment as a bounded subproblem of the overall parsing pipeline. note reattachment receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, the tuned configuration uses a window of 42 units, a stride of 5 units and a confidence floor of 0.2. The implementation of note reattachment follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, memory usage stays below 39 megabytes per worker because intermediate representations are streamed rather than materialized.

In this section we describe note reattachment. The design goal is to make note reattachment reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, annotation guidelines were revised 40 times, and inter annotator agreement settled at kappa equal to 0.1. The cost of note reattachment is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, the tuned configuration uses a window of 38 units, a stride of 24 units and a confidence floor of 0.8. An important design decision concerns the granularity at which note reattachment operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, memory usage stays below 28 megabytes per worker because intermediate representations are streamed rather than materialized.

3.4.2 Column Bottom Routing

We report the behaviour of endnote distinction on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, annotation guidelines were revised 15 times, and inter annotator agreement settled at kappa equal to 0.1. Our first prototype of endnote distinction relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, the tuned configuration uses a window of 13 units, a stride of 8 units and a confidence floor of 0.8. We report the behaviour of endnote distinction on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 3 steps and early stopping patience of 20 epochs. Our first prototype of endnote distinction relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, the tuned configuration uses a window of 9 units, a stride of 19 units and a confidence floor of 0.2. The current version of endnote distinction is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, a single pass over the corpus of 8 documents completes in 4 minutes on eight worker processes without GPU acceleration.

4 Evaluation

Because hierarchy induction interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, the tuned configuration uses a window of 35 units, a stride of 14 units and a confidence floor of 0.8. A recurring

difficulty for hierarchy induction is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 41 steps and early stopping patience of 6 epochs. The current version of hierarchy induction is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, error bars are obtained by bootstrapping 39 resamples of the evaluation set, reported as a 20 percent confidence interval. A recurring difficulty for hierarchy induction is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, annotation guidelines were revised 21 times, and inter annotator agreement settled at kappa equal to 0.7.

4.1 Benchmarks and Metrics

A recurring difficulty for citation linking is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, the default threshold is set to 0.4, chosen on the development split to maximize the harmonic mean of precision and recall. Empirically, citation linking accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, annotation guidelines were revised 12 times, and inter annotator agreement settled at kappa equal to 0.5. We treat citation linking as a bounded subproblem of the overall parsing pipeline. citation linking receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, the tuned configuration uses a window of 6 units, a stride of 9 units and a confidence floor of 0.6. Because citation linking interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 40 steps and early stopping patience of 14 epochs.

4.1.1 Citation Span Detection

An important design decision concerns the granularity at which ambiguous key resolution operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, a single pass over the corpus of 38 documents completes in 23 minutes on eight worker processes without GPU acceleration. The implementation of ambiguous key resolution follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, we cap the candidate list at 40 entries and truncate each entry to 19 characters before scoring. An important design decision concerns the granularity at which ambiguous key resolution operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, the fallback path is triggered for roughly 10 percent of pages, mostly scanned documents with a damaged text layer. The cost of ambiguous key resolution is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, the default threshold is set to 0.8, chosen on the development split to maximize the harmonic mean of precision and recall. We treat ambiguous key resolution as a bounded subproblem of the overall parsing pipeline. ambiguous key resolution receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 13 steps and early stopping patience of 23 epochs.

4.1.2 Bibliography Alignment

The implementation of bibliography alignment follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, memory usage stays below 13 megabytes per worker because intermediate representations are streamed rather than materialized. An important design decision concerns the granularity at which bibliography alignment operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, the tuned configuration uses a window of 7 units, a stride of 24 units and a confidence floor of 0.2. The implementation of bibliography alignment follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, memory usage stays below 17 megabytes per worker because intermediate representations are streamed rather than materialized. In this section we describe bibliography alignment. The design goal is to make bibliography alignment reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, the default threshold is set to 0.8, chosen on the development split to maximize the harmonic mean of precision and recall. Because bibliography alignment interacts with

neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, annotation guidelines were revised 14 times, and inter annotator agreement settled at kappa equal to 0.3.

4.2 Main Results

Our first prototype of language identification relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.4 are treated as meaningful. The cost of language identification is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 3 steps and early stopping patience of 5 epochs. An important design decision concerns the granularity at which language identification operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, we cap the candidate list at 21 entries and truncate each entry to 27 characters before scoring. The cost of language identification is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, memory usage stays below 23 megabytes per worker because intermediate representations are streamed rather than materialized.

4.2.1 Script Level Detection

We treat mixed language segments as a bounded subproblem of the overall parsing pipeline. mixed language segments receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, we cap the candidate list at 22 entries and truncate each entry to 2 characters before scoring. Because mixed language segments interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, annotation guidelines were revised 16 times, and inter annotator agreement settled at kappa equal to 0.5. We evaluated mixed language segments against two reference implementations. The comparison is reported in the accompanying result tables and discussed in the analysis subsection below. Specifically, the tuned configuration uses a window of 26 units, a stride of 25 units and a confidence floor of 0.6. Because mixed language segments interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, annotation guidelines were revised 20 times, and inter annotator agreement settled at kappa equal to 0.3. In this section we describe mixed language segments. The design goal is to make mixed language segments reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, error bars are obtained by bootstrapping 37 resamples of the evaluation set, reported as a 6 percent confidence interval.

4.2.2 Mixed Language Segments

We treat script level detection as a bounded subproblem of the overall parsing pipeline. script level detection receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, error bars are obtained by bootstrapping 13 resamples of the evaluation set, reported as a 13 percent confidence interval. Empirically, script level detection accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, annotation guidelines were revised 31 times, and inter annotator agreement settled at kappa equal to 0.5. We treat script level detection as a bounded subproblem of the overall parsing pipeline. script level detection receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, the tuned configuration uses a window of 9 units, a stride of 20 units and a confidence floor of 0.6. Empirically, script level detection accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, the fallback path is triggered for roughly 11 percent of pages, mostly scanned documents with a damaged text layer. Our first prototype of script level detection relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, we cap the candidate list at 38 entries and truncate each entry to 14 characters before scoring.

The cost of script level detection is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, we cap the candidate list at 13 entries and truncate each entry to 13 characters before scoring. An important design decision concerns the granularity at which script level detection operates. Too coarse a granularity hides local evidence, while too fine a

granularity makes the aggregation step unstable. Specifically, the fallback path is triggered for roughly 31 percent of pages, mostly scanned documents with a damaged text layer. We report the behaviour of script level detection on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, error bars are obtained by bootstrapping 9 resamples of the evaluation set, reported as a 20 percent confidence interval.

4.3 Ablation Studies

We report the behaviour of OCR fallback routing on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, we cap the candidate list at 4 entries and truncate each entry to 25 characters before scoring. In this section we describe OCR fallback routing. The design goal is to make OCR fallback routing reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, annotation guidelines were revised 26 times, and inter annotator agreement settled at kappa equal to 0.5. We report the behaviour of OCR fallback routing on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, error bars are obtained by bootstrapping 16 resamples of the evaluation set, reported as a 7 percent confidence interval. An important design decision concerns the granularity at which OCR fallback routing operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, annotation guidelines were revised 14 times, and inter annotator agreement settled at kappa equal to 0.3.

4.3.1 Text Layer Quality Scoring

Empirically, text layer quality scoring accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.8 are treated as meaningful. We evaluated text layer quality scoring against two reference implementations. The comparison is reported in the accompanying result tables and discussed in the analysis subsection below. Specifically, annotation guidelines were revised 32 times, and inter annotator agreement settled at kappa equal to 0.1. The current version of text layer quality scoring is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, the tuned configuration uses a window of 26 units, a stride of 4 units and a confidence floor of 0.2. We evaluated text layer quality scoring against two reference implementations. The comparison is reported in the accompanying result tables and discussed in the analysis subsection below. Specifically, the fallback path is triggered for roughly 4 percent of pages, mostly scanned documents with a damaged text layer. We report the behaviour of text layer quality scoring on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, the default threshold is set to 0.4, chosen on the development split to maximize the harmonic mean of precision and recall.

4.3.2 Engine Selection Policy

We treat post OCR repair as a bounded subproblem of the overall parsing pipeline. post OCR repair receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.8 are treated as meaningful. Empirically, post OCR repair accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, a single pass over the corpus of 39 documents completes in 15 minutes on eight worker processes without GPU acceleration. We treat post OCR repair as a bounded subproblem of the overall parsing pipeline. post OCR repair receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, error bars are obtained by bootstrapping 17 resamples of the evaluation set, reported as a 28 percent confidence interval. Because post OCR repair interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, a single pass over the corpus of 19 documents completes in 27 minutes on eight worker processes without GPU acceleration. In this section we describe post OCR repair. The design goal is to make post OCR repair reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, the default threshold is set to 0.4, chosen on the development split to maximize the harmonic mean of precision and recall.

4.4 Error Analysis

A recurring difficulty for document ingestion is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, the default threshold is set to 0.4, chosen on the development split to maximize the harmonic mean of precision and recall. Because document ingestion interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, memory usage stays below 13 megabytes per worker because intermediate representations are streamed rather than materialized. We evaluated document ingestion against two reference implementations. The comparison is reported in the accompanying result tables and discussed in the analysis subsection below. Specifically, the default threshold is set to 0.6, chosen on the development split to maximize the harmonic mean of precision and recall. Empirically, document ingestion accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, the fallback path is triggered for roughly 33 percent of pages, mostly scanned documents with a damaged text layer.

4.4.1 File Format Normalization

An important design decision concerns the granularity at which encoding repair operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, the default threshold is set to 0.2, chosen on the development split to maximize the harmonic mean of precision and recall. The implementation of encoding repair follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, memory usage stays below 40 megabytes per worker because intermediate representations are streamed rather than materialized. An important design decision concerns the granularity at which encoding repair operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.8 are treated as meaningful. The implementation of encoding repair follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, the fallback path is triggered for roughly 20 percent of pages, mostly scanned documents with a damaged text layer. We treat encoding repair as a bounded subproblem of the overall parsing pipeline. encoding repair receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.6 are treated as meaningful.

Because encoding repair interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.2 are treated as meaningful. We treat encoding repair as a bounded subproblem of the overall parsing pipeline. encoding repair receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, memory usage stays below 40 megabytes per worker because intermediate representations are streamed rather than materialized. Empirically, encoding repair accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, the tuned configuration uses a window of 10 units, a stride of 17 units and a confidence floor of 0.8.

Table 4.4.1: encoding repair.

System	reference field F1 (%)	delta
Rule based pipeline	89.6	-
Layout model + heuristics	91.4	+1.8
Proposed pipeline	93.1	+3.5

4.4.2 Page Rasterization

A recurring difficulty for text layer validation is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, the default threshold is set to 0.2, chosen on the development split to maximize the harmonic mean of precision and recall. The current version of text layer validation is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 7 steps and early stopping patience of 9 epochs. We treat text layer validation as a bounded subproblem of the overall parsing pipeline. text layer validation receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on.

Specifically, the default threshold is set to 0.8, chosen on the development split to maximize the harmonic mean of precision and recall. Because text layer validation interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, the fallback path is triggered for roughly 3 percent of pages, mostly scanned documents with a damaged text layer. In this section we describe text layer validation. The design goal is to make text layer validation reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, the tuned configuration uses a window of 6 units, a stride of 14 units and a confidence floor of 0.6.

5 Deployment and Conclusion

The cost of table extraction is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.8 are treated as meaningful. In this section we describe table extraction. The design goal is to make table extraction reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, the fallback path is triggered for roughly 28 percent of pages, mostly scanned documents with a damaged text layer. The implementation of table extraction follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.2 are treated as meaningful. An important design decision concerns the granularity at which table extraction operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 32 steps and early stopping patience of 16 epochs.

5.1 Throughput and Latency

Empirically, layout segmentation accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, the default threshold is set to 0.8, chosen on the development split to maximize the harmonic mean of precision and recall. We treat layout segmentation as a bounded subproblem of the overall parsing pipeline. layout segmentation receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, annotation guidelines were revised 20 times, and inter annotator agreement settled at kappa equal to 0.1. Empirically, layout segmentation accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.2 are treated as meaningful. We treat layout segmentation as a bounded subproblem of the overall parsing pipeline. layout segmentation receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, a single pass over the corpus of 32 documents completes in 26 minutes on eight worker processes without GPU acceleration.

5.1.1 Region Proposal Generation

The current version of block clustering is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, the default threshold is set to 0.6, chosen on the development split to maximize the harmonic mean of precision and recall. We evaluated block clustering against two reference implementations. The comparison is reported in the accompanying result tables and discussed in the analysis subsection below. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 36 steps and early stopping patience of 11 epochs. Empirically, block clustering accounts for a measurable share of end-to-end errors. We therefore instrument it with counters for rejected inputs, ambiguous outputs, and fallback invocations. Specifically, error bars are obtained by bootstrapping 38 resamples of the evaluation set, reported as a 13 percent confidence interval. A recurring difficulty for block clustering is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 32 steps and early stopping patience of 22 epochs. The implementation of block clustering follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, the default threshold is set to 0.2, chosen on the development split to maximize the harmonic mean of precision and recall.

5.1.2 Column Boundary Estimation

In this section we describe column boundary estimation. The design goal is to make column boundary estimation reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, the default threshold is set to 0.6, chosen on the development split to maximize the harmonic mean of precision and recall. The cost of column boundary estimation is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, memory usage stays below 19 megabytes per worker because intermediate representations are streamed rather than materialized. In this section we describe column boundary estimation. The design goal is to make column boundary estimation reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, error bars are obtained by bootstrapping 5 resamples of the evaluation set, reported as a 9 percent confidence interval. We report the behaviour of column boundary estimation on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, the fallback path is triggered for roughly 39 percent of pages, mostly scanned documents with a damaged text layer. A recurring difficulty for column boundary estimation is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.2 are treated as meaningful.

5.2 Resource Budgets

An important design decision concerns the granularity at which reading order recovery operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, the tuned configuration uses a window of 25 units, a stride of 23 units and a confidence floor of 0.8. We report the behaviour of reading order recovery on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, a single pass over the corpus of 27 documents completes in 23 minutes on eight worker processes without GPU acceleration. An important design decision concerns the granularity at which reading order recovery operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, the default threshold is set to 0.2, chosen on the development split to maximize the harmonic mean of precision and recall. The cost of reading order recovery is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, the fallback path is triggered for roughly 39 percent of pages, mostly scanned documents with a damaged text layer.

5.2.1 Intra-Column Ordering

In this section we describe cross-column stitching. The design goal is to make cross-column stitching reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, memory usage stays below 10 megabytes per worker because intermediate representations are streamed rather than materialized. The implementation of cross-column stitching follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, the tuned configuration uses a window of 20 units, a stride of 15 units and a confidence floor of 0.8. An important design decision concerns the granularity at which cross-column stitching operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, a single pass over the corpus of 38 documents completes in 9 minutes on eight worker processes without GPU acceleration. We report the behaviour of cross-column stitching on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, we cap the candidate list at 16 entries and truncate each entry to 10 characters before scoring. We treat cross-column stitching as a bounded subproblem of the overall parsing pipeline. cross-column stitching receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, annotation guidelines were revised 33 times, and inter annotator agreement settled at kappa equal to 0.5.

5.2.2 Cross-Column Stitching

An important design decision concerns the granularity at which intra-column ordering operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, annotation guidelines were revised 33 times, and inter annotator agreement settled at kappa equal to 0.1. The cost of intra-column ordering is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, the tuned configuration uses a

window of 27 units, a stride of 26 units and a confidence floor of 0.8. An important design decision concerns the granularity at which intra-column ordering operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, a single pass over the corpus of 29 documents completes in 5 minutes on eight worker processes without GPU acceleration. The cost of intra-column ordering is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, the default threshold is set to 0.2, chosen on the development split to maximize the harmonic mean of precision and recall. We evaluated intra-column ordering against two reference implementations. The comparison is reported in the accompanying result tables and discussed in the analysis subsection below. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 34 steps and early stopping patience of 19 epochs.

The current version of intra-column ordering is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, annotation guidelines were revised 33 times, and inter annotator agreement settled at kappa equal to 0.1. A recurring difficulty for intra-column ordering is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, the default threshold is set to 0.8, chosen on the development split to maximize the harmonic mean of precision and recall. The current version of intra-column ordering is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 29 steps and early stopping patience of 5 epochs.

5.3 Monitoring

Our first prototype of heading detection relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, we cap the candidate list at 28 entries and truncate each entry to 15 characters before scoring. We report the behaviour of heading detection on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, a single pass over the corpus of 10 documents completes in 8 minutes on eight worker processes without GPU acceleration. An important design decision concerns the granularity at which heading detection operates. Too coarse a granularity hides local evidence, while too fine a granularity makes the aggregation step unstable. Specifically, we cap the candidate list at 40 entries and truncate each entry to 6 characters before scoring. The implementation of heading detection follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 14 steps and early stopping patience of 10 epochs.

5.3.1 Typographic Feature Design

In this section we describe typographic feature design. The design goal is to make typographic feature design reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, a single pass over the corpus of 34 documents completes in 2 minutes on eight worker processes without GPU acceleration. The cost of typographic feature design is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, the tuned configuration uses a window of 36 units, a stride of 26 units and a confidence floor of 0.4. In this section we describe typographic feature design. The design goal is to make typographic feature design reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, the fallback path is triggered for roughly 38 percent of pages, mostly scanned documents with a damaged text layer. The cost of typographic feature design is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, the tuned configuration uses a window of 40 units, a stride of 19 units and a confidence floor of 0.6. We treat typographic feature design as a bounded subproblem of the overall parsing pipeline. typographic feature design receives a normalized page representation and emits a typed structure together with a confidence value that later stages may abstain on. Specifically, a single pass over the corpus of 25 documents completes in 25 minutes on eight worker processes without GPU acceleration.

5.3.2 Numbered Heading Matching

The implementation of negative example mining follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, annotation guidelines were revised 9 times, and inter annotator agreement settled at kappa equal to 0.5. Our first

prototype of negative example mining relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.4 are treated as meaningful. The cost of negative example mining is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 29 steps and early stopping patience of 16 epochs. Our first prototype of negative example mining relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, the default threshold is set to 0.6, chosen on the development split to maximize the harmonic mean of precision and recall. The current version of negative example mining is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, the fallback path is triggered for roughly 18 percent of pages, mostly scanned documents with a damaged text layer.

5.4 Summary and Future Work

In this section we describe hierarchy induction. The design goal is to make hierarchy induction reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, the default threshold is set to 0.8, chosen on the development split to maximize the harmonic mean of precision and recall. We report the behaviour of hierarchy induction on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, we fixed the learning rate schedule to cosine decay with a warmup of 29 steps and early stopping patience of 19 epochs. Our first prototype of hierarchy induction relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, the tuned configuration uses a window of 35 units, a stride of 17 units and a confidence floor of 0.2. The cost of hierarchy induction is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, a single pass over the corpus of 9 documents completes in 25 minutes on eight worker processes without GPU acceleration.

5.4.1 Depth Estimation

In this section we describe outline validation. The design goal is to make outline validation reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, a single pass over the corpus of 18 documents completes in 4 minutes on eight worker processes without GPU acceleration. The implementation of outline validation follows three principles: determinism, observability, and graceful degradation. None of the three is optional once the pipeline is deployed over heterogeneous corpora. Specifically, error bars are obtained by bootstrapping 28 resamples of the evaluation set, reported as a 2 percent confidence interval. Our first prototype of outline validation relied on hand written heuristics. Replacing them with a calibrated scoring function reduced variance across publishers without hurting the average case. Specifically, a single pass over the corpus of 38 documents completes in 11 minutes on eight worker processes without GPU acceleration. The cost of outline validation is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, the default threshold is set to 0.4, chosen on the development split to maximize the harmonic mean of precision and recall. We evaluated outline validation against two reference implementations. The comparison is reported in the accompanying result tables and discussed in the analysis subsection below. Specifically, the fallback path is triggered for roughly 9 percent of pages, mostly scanned documents with a damaged text layer.

The current version of outline validation is deliberately conservative: when the evidence is weak it emits a low confidence record instead of guessing, and the pipeline treats such records as soft failures. Specifically, annotation guidelines were revised 18 times, and inter annotator agreement settled at kappa equal to 0.3. A recurring difficulty for outline validation is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, we cap the candidate list at 28 entries and truncate each entry to 2 characters before scoring. Because outline validation interacts with neighbouring stages, we specified an explicit interface contract: every field carries a provenance tag so that errors can be traced back to the stage that produced them. Specifically, annotation guidelines were revised 38 times, and inter annotator agreement settled at kappa equal to 0.5.

5.4.2 Parent Child Assignment

Our first prototype of level smoothing relied on hand written heuristics. Replacing them with a calibrated scoring

function reduced variance across publishers without hurting the average case. Specifically, the fallback path is triggered for roughly 17 percent of pages, mostly scanned documents with a damaged text layer. We report the behaviour of level smoothing on adversarial inputs separately, since averages over the full corpus hide the tail that matters most for robustness. Specifically, the default threshold is set to 0.6, chosen on the development split to maximize the harmonic mean of precision and recall. In this section we describe level smoothing. The design goal is to make level smoothing reliable on documents that were never seen during development, and to keep its failure modes predictable for downstream consumers. Specifically, memory usage stays below 21 megabytes per worker because intermediate representations are streamed rather than materialized. The cost of level smoothing is dominated by the number of candidate regions rather than by the number of pages, which makes the per-document cost essentially linear in practice. Specifically, the ablation removes one component at a time while holding all random seeds fixed, so differences above 0.4 are treated as meaningful. A recurring difficulty for level smoothing is the tension between recall and precision on atypical templates. We resolved it by deferring borderline decisions to the verification stage. Specifically, the fallback path is triggered for roughly 42 percent of pages, mostly scanned documents with a damaged text layer.

References

[1] Author D Tanaka, J. Anderson. Calibrated layout segmentation for heterogeneous scholarly corpora. In Proceedings of the 2016 ACM Symposium on Document Engineering, pages 107-118, 2016.

[2] Author B Nowak, M. Eriksen. Scalable reading order recovery for heterogeneous scholarly corpora. In Proceedings of the 2017 Joint Conference on Digital Libraries, pages 114-125, 2017.

[3] Author D Tanaka, M. Costa. Robust heading detection for heterogeneous scholarly corpora. In Proceedings of the 2018 International Conference on Document Analysis, pages 121-132, 2018.

[4] Author A Nowak, R. Eriksen. Calibrated hierarchy induction for heterogeneous scholarly corpora. In Proceedings of the 2019 Annual Meeting of the Association for Computational Linguistics, pages 128-139, 2019.

[5] Author C Tanaka, K. Costa. Hierarchical table extraction for heterogeneous scholarly corpora. In Proceedings of the 2020 ACM Symposium on Document Engineering, pages 135-146, 2020.

[6] Author A Nowak, J. Anderson. Hierarchical formula recognition for heterogeneous scholarly corpora. In Proceedings of the 2021 Joint Conference on Digital Libraries, pages 142-153, 2021.

[7] Author C Tanaka, R. Bergman. Robust reference parsing for heterogeneous scholarly corpora. In Proceedings of the 2022 International Conference on Document Analysis, pages 149-160, 2022.

[8] Author D Smith, K. Costa. Efficient metadata reconciliation for heterogeneous scholarly corpora. In Proceedings of the 2023 Annual Meeting of the Association for Computational Linguistics, pages 156-167, 2023.

[9] Author B Okafor, L. Costa. Efficient cross-page boilerplate filtering for heterogeneous scholarly corpora. In Proceedings of the 2024 ACM Symposium on Document Engineering, pages 163-174, 2024.

[10] Author B Smith, K. Dubois. Efficient multi-column flow analysis for heterogeneous scholarly corpora. In Proceedings of the 2025 Joint Conference on Digital Libraries, pages 170-181, 2025.

[11] Author D Kumar, R. Dubois. Robust section boundary scoring for heterogeneous scholarly corpora. In Proceedings of the 2015 International Conference on Document Analysis, pages 177-188, 2015.

[12] Author B Nowak, J. Bergman. Efficient outline normalization for heterogeneous scholarly corpora. In Proceedings of the 2016 Annual Meeting of the Association for Computational Linguistics, pages 184-195, 2016.

[13] Author D Kumar, M. Bergman. Scalable long-context chunking for heterogeneous scholarly corpora. In Proceedings of the 2017 ACM Symposium on Document Engineering, pages 191-202, 2017.

[14] Author A Nowak, J. Bergman. Calibrated semantic section typing for heterogeneous scholarly corpora. In Proceedings of the 2018 Joint Conference on Digital Libraries, pages 198-209, 2018.

[15] Author C Tanaka, K. Eriksen. Scalable figure caption association for heterogeneous scholarly corpora. In Proceedings of the 2019 International Conference on Document Analysis, pages 205-216, 2019.

[16] Author A Ferreira, K. Costa. Hierarchical footnote separation for heterogeneous scholarly corpora. In Proceedings of the 2020 Annual Meeting of the Association for Computational Linguistics, pages 212-223, 2020.

[17] Author C Kumar, K. Dubois. Calibrated citation linking for heterogeneous scholarly corpora. In Proceedings of the 2021 ACM Symposium on Document Engineering, pages 219-230, 2021.

[18] Author D Smith, M. Eriksen. Robust language identification for heterogeneous scholarly corpora. In Proceedings of the 2022 Joint Conference on Digital Libraries, pages 226-237, 2022.

[19] Author B Tanaka, R. Eriksen. Efficient OCR fallback routing for heterogeneous scholarly corpora. In Proceedings of the 2023 International Conference on Document Analysis, pages 233-244, 2023.

[20] Author C Ferreira, K. Dubois. Scalable document ingestion for heterogeneous scholarly corpora. In Proceedings of the 2024 Annual Meeting of the Association for Computational Linguistics, pages 240-251, 2024.

[21] Author A Kumar, J. Bergman. Calibrated layout segmentation for heterogeneous scholarly corpora. In Proceedings of the 2025 ACM Symposium on Document Engineering, pages 247-258, 2025.

[22] Author C Smith, K. Costa. Hierarchical reading order recovery for heterogeneous scholarly corpora. In Proceedings of the 2015 Joint Conference on Digital Libraries, pages 254-265, 2015.

[23] Author A Okafor, K. Costa. Calibrated heading detection for heterogeneous scholarly corpora. In Proceedings of the 2016 International Conference on Document Analysis, pages 261-272, 2016.

[24] Author D Ferreira, K. Eriksen. Calibrated hierarchy induction for heterogeneous scholarly corpora. In Proceedings of the 2017 Annual Meeting of the Association for Computational Linguistics, pages 268-279, 2017.

[25] Author B Tanaka, M. Eriksen. Calibrated table extraction for heterogeneous scholarly corpora. In Proceedings of the 2018 ACM Symposium on Document Engineering, pages 275-286, 2018.

[26] Author D Ferreira, R. Bergman. Hierarchical formula recognition for heterogeneous scholarly corpora. In Proceedings of the 2019 Joint Conference on Digital Libraries, pages 282-293, 2019.

[27] Author B Okafor, L. Dubois. Calibrated reference parsing for heterogeneous scholarly corpora. In Proceedings of the

2020 International Conference on Document Analysis, pages 289-300, 2020.

[28] Author A Smith, K. Dubois. Hierarchical metadata reconciliation for heterogeneous scholarly corpora. In Proceedings of the 2021 Annual Meeting of the Association for Computational Linguistics, pages 296-307, 2021.

[29] Author C Tanaka, M. Eriksen. Calibrated cross-page boilerplate filtering for heterogeneous scholarly corpora. In Proceedings of the 2022 ACM Symposium on Document Engineering, pages 303-314, 2022.

[30] Author D Okafor, R. Eriksen. Robust multi-column flow analysis for heterogeneous scholarly corpora. In Proceedings of the 2023 International Conference on Document Analysis, pages 310-321, 2023.

[31] Author B Smith, M. Costa. Robust section boundary scoring for heterogeneous scholarly corpora. In Proceedings of the 2024 Joint Conference on Digital Libraries, pages 317-328, 2024.

[32] Author D Okafor, K. Dubois. Robust outline normalization for heterogeneous scholarly corpora. In Proceedings of the 2025 ACM Symposium on Document Engineering, pages 324-335, 2025.

[33] Author B Ferreira, R. Costa. Hierarchical long-context chunking for heterogeneous scholarly corpora. In Proceedings of the 2015 Annual Meeting of the Association for Computational Linguistics, pages 331-342, 2015.

[34] Author C Okafor, M. Costa. Hierarchical semantic section typing for heterogeneous scholarly corpora. In Proceedings of the 2016 International Conference on Document Analysis, pages 338-349, 2016.

[35] Author A Nowak, K. Anderson. Scalable figure caption association for heterogeneous scholarly corpora. In Proceedings of the 2017 Joint Conference on Digital Libraries, pages 345-356, 2017.

[36] Author C Tanaka, L. Bergman. Efficient footnote separation for heterogeneous scholarly corpora. In Proceedings of the 2018 ACM Symposium on Document Engineering, pages 352-363, 2018.

[37] Author A Nowak, L. Dubois. Calibrated citation linking for heterogeneous scholarly corpora. In Proceedings of the 2019 Annual Meeting of the Association for Computational Linguistics, pages 359-370, 2019.

[38] Author B Okafor, J. Costa. Robust language identification for heterogeneous scholarly corpora. In Proceedings of the 2020 International Conference on Document Analysis, pages 366-377, 2020.

[39] Author D Smith, M. Eriksen. Robust OCR fallback routing for heterogeneous scholarly corpora. In Proceedings of the 2021 Joint Conference on Digital Libraries, pages 373-384, 2021.

[40] Author A Smith, J. Dubois. Robust document ingestion for heterogeneous scholarly corpora. In Proceedings of the 2022 Annual Meeting of the Association for Computational Linguistics, pages 380-391, 2022.

[41] Author C Okafor, M. Anderson. Hierarchical layout segmentation for heterogeneous scholarly corpora. In Proceedings of the 2023 ACM Symposium on Document Engineering, pages 387-398, 2023.

[42] Author A Ferreira, J. Anderson. Calibrated reading order recovery for heterogeneous scholarly corpora. In Proceedings of the 2024 Joint Conference on Digital Libraries, pages 394-405, 2024.

[43] Author C Tanaka, K. Dubois. Efficient heading detection for heterogeneous scholarly corpora. In Proceedings of the 2025 International Conference on Document Analysis, pages 401-412, 2025.

[44] Author B Smith, K. Bergman. Efficient hierarchy induction for heterogeneous scholarly corpora. In Proceedings of the 2015 Annual Meeting of the Association for Computational Linguistics, pages 408-419, 2015.

[45] Author D Kumar, R. Eriksen. Robust table extraction for heterogeneous scholarly corpora. In Proceedings of the 2016 ACM Symposium on Document Engineering, pages 415-426, 2016.

[46] Author B Nowak, J. Eriksen. Efficient formula recognition for heterogeneous scholarly corpora. In Proceedings of the 2017 Joint Conference on Digital Libraries, pages 422-433, 2017.

[47] Author D Tanaka, M. Dubois. Calibrated reference parsing for heterogeneous scholarly corpora. In Proceedings of the 2018 International Conference on Document Analysis, pages 429-440, 2018.

[48] Author C Smith, M. Anderson. Robust metadata reconciliation for heterogeneous scholarly corpora. In Proceedings of the 2019 Annual Meeting of the Association for Computational Linguistics, pages 436-447, 2019.